

ADS Exam Cheatsheet

1. Data Structures — Time Complexities

Arrays

Contiguous memory. Fixed size (static) or resizable (dynamic/ArrayList). Index-based $O(1)$ access.

Operation	Unsorted Array	Sorted Array	Dynamic Array (ArrayList)
Access by index	$O(1)$	$O(1)$	$O(1)$
Search	$O(n)$	$O(\log n)$ binary search	$O(n)$
Insert at end	$O(1)$	$O(n)$ shift	$O(1)$ amortized
Insert at index	$O(n)$	$O(n)$	$O(n)$
Delete at index	$O(n)$	$O(n)$	$O(n)$
Delete at end	$O(1)$	$O(1)$	$O(1)$
Space	$O(n)$	$O(n)$	$O(n)$

Singly Linked List

Each node has data + next pointer. No random access. Must traverse from head.

Operation	Time	Notes
Access by index	$O(n)$	Must traverse
Search	$O(n)$	Linear scan
Insert at head	$O(1)$	Update head pointer
Insert at tail	$O(1)$ w/ tail ref, $O(n)$ without	Keep tail reference!
Insert after node	$O(1)$	Given reference to node
Delete head	$O(1)$	Update head
Delete given node	$O(n)$	Need predecessor — must traverse
Space	$O(n)$	Extra pointer per node

Doubly Linked List

Each node has prev + next pointers. Can traverse both directions. Sentinels (header/trailer) simplify edge cases.

Operation	Time	Notes
Access by index	$O(n)$	Still must traverse
Search	$O(n)$	Can search from either end
Insert at head/tail	$O(1)$	With sentinels: trivial
Insert before/after node	$O(1)$	Given reference to node
Delete given node	$O(1)$	Has prev pointer — no traversal needed!
Delete head/tail	$O(1)$	Direct access via sentinels
Space	$O(n)$	Two pointers per node

Key advantage over singly linked: delete a given node in $O(1)$ because you have the prev pointer.

Circular Linked List

Last node points back to first (singly or doubly). Often uses a 'cursor' pointing to the current node. Good for round-robin scheduling.

Operation	Time	Notes
Rotate	$O(1)$	Move cursor to next
Insert after cursor	$O(1)$	
Delete after cursor	$O(1)$	Remove cursor.next
Search	$O(n)$	Traverse full cycle
Space	$O(n)$	

1. Data Structures — continued

Stack (LIFO)

Last In, First Out. Can be implemented with array or linked list.

Operation	Time	Notes
push(e)	O(1)	Add to top (amortized if array-based)
pop()	O(1)	Remove from top
top() / peek()	O(1)	View top without removing
size(), isEmpty()	O(1)	
Space	O(n)	

Array-based: use index t as top pointer. **Linked-list-based:** push/pop at head.

Queue (FIFO)

First In, First Out. Array-based uses circular array (mod N) to avoid shifting.

Operation	Time	Notes
enqueue(e)	O(1)	Add to rear
dequeue()	O(1)	Remove from front
first() / peek()	O(1)	View front
size(), isEmpty()	O(1)	

Circular array: front index f , rear = $(f + \text{size}) \% N$. When full, can resize (copy to new array).

Double-Ended Queue (Deque)

Insert and remove at both ends. Generalizes both stack and queue.

Operation	Time	Notes
addFirst(e) / addLast(e)	O(1)	Insert at front/rear
removeFirst() / removeLast()	O(1)	Remove from front/rear
first() / last()	O(1)	Peek at either end
size(), isEmpty()	O(1)	

Impl: Doubly linked list (natural fit) or circular array. Java: ArrayDeque, LinkedList.

Iterators

Design pattern for traversing a collection without exposing internal structure. In Java: Iterable + Iterator interfaces.

Method	Description
hasNext()	Returns true if more elements
next()	Returns next element, advances cursor
remove()	Removes last returned element (optional)

Iterable<E> has iterator() method. Enables for-each loops. **Snapshot iterator:** copies collection at creation — not affected by changes. **Lazy iterator:** reflects live state — may throw ConcurrentModificationException if modified.

Priority Queue

Elements have keys (priorities). Min or max extracted first. Typically implemented with a binary heap.

Operation	Unsorted List	Sorted List	Binary Heap
insert(k,v)	O(1)	O(n)	O(log n)
min() / peek()	O(n)	O(1)	O(1)
removeMin()	O(n)	O(1)	O(log n)
size(), isEmpty()	O(1)	O(1)	O(1)
Space	O(n)	O(n)	O(n)

Binary heap: complete binary tree stored in array. Parent at i , children at $2i+1$ and $2i+2$. **Upheap** after insert (bubble up). **Downheap** after removeMin (bubble down). **Heapify** (bottom-up construction): O(n).

Maps

Key-value pairs. Keys are unique.

Operation	Unsorted List Map	Hash Map (avg)	Hash Map (worst)	Sorted Map (tree)
get(k)	O(n)	O(1)	O(n)	O(log n)
put(k,v)	O(n)	O(1)	O(n)	O(log n)
remove(k)	O(n)	O(1)	O(n)	O(log n)
size(), isEmpty()	O(1)	O(1)	O(1)	O(1)
Space	O(n)	O(n)	O(n)	O(n)

Hash map: hash function maps keys to buckets. Load factor = n/N . Collision handling: separate chaining (list per bucket) or open addressing (linear probing). Rehash when load factor exceeds threshold (~ 0.75).

1. Data Structures — Trees

Tree Terminology

Root: top node, no parent. **Internal node:** has at least one child. **Leaf/external:** no children. **Depth:** number of ancestors (root=0). **Height:** max depth of any node. **Subtree:** node + all its descendants.

Binary Tree

Each node has at most 2 children (left, right). **Proper/full:** every node has 0 or 2 children. **Complete:** all levels full except possibly last (filled left to right). **Perfect:** all leaves at same depth.

Property	Formula
Max nodes at depth d	2^d
Max total nodes (height h)	$2^{h+1} - 1$
Min height (n nodes)	$\text{floor}(\log_2 n)$
# external = # internal + 1	For proper binary trees

Tree Traversals

Traversal	Order	Use Case
Pre-order	Node, Left, Right	Copy tree, prefix expression
In-order	Left, Node, Right	BST: sorted order
Post-order	Left, Right, Node	Delete tree, postfix expression
Level-order (BFS)	Level by level (use queue)	Breadth-first search

All traversals: $O(n)$ time.

Binary Search Tree (BST)

Left subtree keys < node key < right subtree keys. In-order traversal gives sorted sequence.

Operation	Average	Worst (degenerate)	Notes
search(k)	$O(\log n)$	$O(n)$	Like binary search on sorted data
insert(k,v)	$O(\log n)$	$O(n)$	Search then insert at leaf position
delete(k)	$O(\log n)$	$O(n)$	Find in-order successor if 2 children
min / max	$O(\log n)$	$O(n)$	Leftmost / rightmost node

Worst case happens when tree degenerates into a linked list (e.g. inserting sorted data). Balanced BSTs (AVL, Red-Black) guarantee $O(\log n)$.

AVL Tree (if covered)

Self-balancing BST. For every node, $|\text{height}(\text{left}) - \text{height}(\text{right})| \leq 1$. After insert/delete, rebalance via rotations.

Operation	Time
search / insert / delete	$O(\log n)$ guaranteed
Rotation	$O(1)$
Height	$O(\log n)$ always

Rotations: single (LL, RR) or double (LR, RL). Trinode restructuring.

2. Sorting Algorithms

Comparison Overview

Algorithm	Best	Average	Worst	Space	Stable?	Method
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No	Select minimum, swap
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	Insert into sorted prefix
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes	Divide & merge
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No	Partition around pivot
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	No	Build heap, extract max
PQ Sort (unsorted)	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(n)$	Yes	Insert all, removeMin n times
PQ Sort (heap)	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	No	Same as heap sort variant

Selection Sort

Find the minimum in the unsorted region, swap it to the front. Repeat. Always $O(n^2)$ comparisons regardless of input. **Not stable** (swaps can change relative order of equal elements).

```
for i = 0 to n-2:
    minIdx = i
    for j = i+1 to n-1:
        if A[j] < A[minIdx]: minIdx = j
    swap A[i], A[minIdx]
```

Insertion Sort

Take next element, insert into correct position in sorted prefix by shifting. **Best case $O(n)$** when already sorted. **Stable**. Great for small or nearly-sorted data.

```
for i = 1 to n-1:
    key = A[i], j = i-1
    while j >= 0 and A[j] > key:
        A[j+1] = A[j], j--
    A[j+1] = key
```

Merge Sort

Divide array in half, recursively sort each half, merge sorted halves. Always $O(n \log n)$. **Stable**. Needs $O(n)$ extra space for merging.

```
mergeSort(A, lo, hi):
    if lo >= hi: return
    mid = (lo+hi)/2
    mergeSort(A, lo, mid)
    mergeSort(A, mid+1, hi)
    merge(A, lo, mid, hi)
```

Merge step: use two pointers, compare heads, copy smaller to temp array. $O(n)$ per level, $O(\log n)$ levels = $O(n \log n)$ total.

Quick Sort

Pick pivot, partition array into [$< \text{pivot}$, pivot , $> \text{pivot}$], recurse on partitions. **Worst $O(n^2)$** when pivot is always min/max (e.g. sorted input with first-element pivot). Average $O(n \log n)$. **Not stable**. In-place ($O(\log n)$ stack space).

```
quickSort(A, lo, hi):
    if lo >= hi: return
    p = partition(A, lo, hi)
    quickSort(A, lo, p-1)
    quickSort(A, p+1, hi)
```

Pivot strategies: first element (bad for sorted data), random (good average), median-of-three (robust). **In practice, fastest general-purpose sort.**

Heap Sort

Build a max-heap from the array (bottom-up $O(n)$), then repeatedly extract max and place at end. Always $O(n \log n)$. **Not stable**. In-place $O(1)$ extra space.

```
1. Build max-heap (heapify):  $O(n)$ 
2. For i = n-1 down to 1:
    swap A[0] with A[i]
    downheap on A[0..i-1]
```

Priority Queue Sort

Insert all elements into a PQ, then removeMin n times. The PQ implementation determines the complexity:

PQ Implementation	Insert phase	Remove phase	Total	Equivalent to
Unsorted list	$O(n)$ total	$O(n^2)$ total	$O(n^2)$	Selection Sort
Sorted list	$O(n^2)$ total	$O(n)$ total	$O(n^2)$	Insertion Sort
Heap	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	Heap Sort

3. Sorting Concepts & Key Formulas

Stable vs Unstable Sorting

A sorting algorithm is **stable** if elements with equal keys maintain their original relative order after sorting.

Stable	Unstable
Insertion Sort	Selection Sort
Merge Sort	Quick Sort
PQ Sort (unsorted list)	Heap Sort

Why it matters: sorting by multiple criteria. Sort by secondary key first, then stable-sort by primary key preserves secondary ordering for equal primary keys.

Comparison-Based Sorting Lower Bound

Any comparison-based sorting algorithm requires at least $O(n \log n)$ comparisons in the worst case. This is proven via decision tree argument: $n!$ leaves, height $\geq \log_2(n!) = O(n \log n)$. **Merge Sort and Heap Sort are asymptotically optimal.**

Key Complexity Classes

Notation	Name	Example
$O(1)$	Constant	Array access, push/pop on stack
$O(\log n)$	Logarithmic	Binary search, balanced BST ops
$O(n)$	Linear	Linear search, traversal
$O(n \log n)$	Linearithmic	Merge sort, heap sort
$O(n^2)$	Quadratic	Insertion sort, selection sort (worst)

Heap Formulas (Array-Based)

Relationship	Formula (0-indexed)
Parent of node i	$(i - 1) / 2$
Left child of node i	$2i + 1$
Right child of node i	$2i + 2$
Last internal node	$n/2 - 1$
Heap property (min-heap)	$A[\text{parent}] \leq A[\text{child}]$

Hash Table Notes

Hash function: maps key to integer (hashCode in Java), then compress: $h(k) \bmod N$. **Load factor** $\lambda = n/N$. **Separate chaining:** each bucket is a list, expected $O(1 + \lambda)$ per op. **Open addressing / linear probing:** find next empty slot, clustering degrades performance. **Rehash** when $\lambda > \text{threshold}$ (double capacity, reinsert all).

Amortized Analysis Reminder

Dynamic array (ArrayList) doubling: individual push can be $O(n)$ when resizing, but amortized over n pushes is $O(1)$ per push. Total cost of n pushes: $O(n)$. Think of it as 'spreading' the expensive resize over the cheap operations.

Good luck on your exam!